

Opis Twojego kodu

HinduskieRomanse

*Materiał dla osoby, która ma napisać podsumowanie projektu.
Opis dotyczy wyłącznie Twojej warstwy kodu i tego, jak ona działa.*

1. Co było Twoją częścią

Z całej historii projektu wynika, że Twoja rola polegała na zbudowaniu wspólnej warstwy danych i infrastruktury. Nie pisałeś głównej implementacji sieci neuronowej ani całej pętli eksperymentalnej dla SSN. Twoim zadaniem było przygotowanie takiego środowiska, żeby inni mogli bezpiecznie trenować modele, porównywać wyniki i nie psuć struktury projektu.

- w części 1: konfiguracja tasków, przygotowanie danych, preprocessing, zapis gotowych splitów, EDA, test warstwy danych;
- w części 2: wspólny opis eksperymentów UM, rejestr modeli, metryki, runner eksperymentów i prosty punkt wejścia z terminala;
- w praktyce: zbudowałeś fundament, na którym pracowali pozostali autorzy.

2. Mapa plików, które opisuje ten dokument

- `src/config.py`
- `src/data_loader.py`
- `src/run_data_prep.py`
- `src/eda.py`
- `src/ml_config.py`
- `src/ml_models.py`
- `src/ml_metrics.py`
- `src/ml_experiments.py`
- `run_um_experiments.py`

Poniżej każdy plik jest opisany blokami. Nie chodzi o tłumaczenie każdej linijki, tylko o zrozumienie po co dany blok istnieje, jaką decyzję projektową realizuje i co daje reszcie projektu.

3. Część 1 - SSN

3.1 `src/config.py`

Ten plik ustawia podstawowe stałe projektu i definiuje taski. To jest centralne miejsce konfiguracji. Dzięki temu reszta kodu nie musi ręcznie wpisywać nazw targetów, ścieżek czy reguł dla klasyfikacji i regresji.

```

from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parents[1]
DATA_DIR = PROJECT_ROOT / "data"
RAW_DATA_PATH = DATA_DIR / "raw" / "marriage_data_india.csv"
PROCESSED_DATA_DIR = DATA_DIR / "processed"
RESULTS_DIR = PROJECT_ROOT / "results"
RANDOM_STATE = 42
DEFAULT_TEST_SIZE = 0.20

```

Pierwszy blok ustala wspólną strukturę folderów i stałe techniczne. Najważniejsze jest to, że wszystkie kolejne moduły pracują na tych samych ścieżkach i tym samym seedzie.

```

TASK_CONFIGS = {
    "classification_divorce": {
        "problem_type": "classification",
        "target": "Divorce_Status",
        "drop_columns": ["ID"],
        "target_mapping": {"No": 0, "Yes": 1},
        "stratify": True,
    },
    "regression_children": {
        "problem_type": "regression",
        "target": "Children_Count",
        "drop_columns": ["ID"],
        "stratify": False,
    },
}

```

Drugi blok definiuje logikę samych zadań. Dla klasyfikacji wskazujesz target, mapowanie klas i potrzebę stratified splitu. Dla regresji wskazujesz target liczbowy i brak stratyfikacji. To jest bardzo ważne, bo później data_loader działa już tylko na tej konfiguracji.

3.2 src/data_loader.py

To jest najważniejszy plik Twojej części. Cały projekt opiera się na tym, że dane są przygotowywane w jednym miejscu i według jednej reguły.

```

class ProjectDataLoader:
    REQUIRED_COLUMNS = [
        "ID", "Marriage_Type", "Age_at_Marriage", "Gender",
        "Education_Level", "Caste_Match", "Religion", "Parental_Approval",
        "Urban_Rural", "Dowry_Exchanged", "Marital_Satisfaction",
        "Divorce_Status", "Children_Count", "Income_Level",
        "Years_Since_Marriage", "Spouse_Working", "Inter-Caste", "Inter-Religion",
    ]

```

Ten blok mówi, jakie kolumny muszą istnieć w zbiorze. To jest zabezpieczenie przed sytuacją, w której ktoś podmieni plik CSV albo niechcący użyje niepełnych danych.

```

def get_features_and_target(self, df, task_name):
    cfg = self.get_task_config(task_name)
    target_col = cfg["target"]
    drop_columns = set(cfg.get("drop_columns", [])) | {target_col}

    X = df[[c for c in df.columns if c not in drop_columns]].copy()
    y = df[target_col].copy()

    if cfg["problem_type"] == "classification":
        mapping = cfg["target_mapping"]
        y = y.map(mapping).astype(int)

    return X, y, cfg

```

Tutaj kod wybiera cechy wejściowe i target. W praktyce ten blok robi trzy rzeczy: usuwa kolumny, których model nie ma widzieć, wyciąga target do osobnej zmiennej i w przypadku klasyfikacji mapuje klasy tekstowe na liczby.

```

def build_preprocessor(self, X, scale_numeric=True):
    numeric_features = X.select_dtypes(include="number").columns.tolist()
    categorical_features = [c for c in X.columns if c not in numeric_features]

    numeric_transformer = Pipeline(steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ])

    categorical_transformer = Pipeline(steps=[
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", _make_one_hot_encoder()),
    ])

    return ColumnTransformer([
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features),
    ])

```

To jest blok preprocessingu. Najpierw rozdzielasz cechy liczbowe i kategoriyczne, a potem dla każdej grupy budujesz osobną ścieżkę obróbki. Dla liczb: imputacja medianą i skalowanie. Dla kategorii: imputacja najczęstszą wartością i OneHotEncoder. Dzięki temu wszystkie modele dostają już gotową, numeryczną macierz cech.

```

X_train_df, X_test_df, y_train, y_test = train_test_split(
    X, y,
    test_size=test_size,
    random_state=random_state,
    stratify=stratify,
)

preprocessor = self.build_preprocessor(X_train_df, scale_numeric=scale_numeric)

```

```
X_train = preprocessor.fit_transform(X_train_df).astype(np.float32)
X_test = preprocessor.transform(X_test_df).astype(np.float32)
```

To jest najważniejszy moment metodologiczny. Najpierw robisz split train/test, a dopiero później dopasowujesz preprocessor na train. To ogranicza data leakage, czyli sytuację, w której informacje ze zbioru testowego przeciekają do procesu uczenia.

```
metadata = {
    "task_name": task_name,
    "problem_type": cfg["problem_type"],
    "target": cfg["target"],
    "input_dim": int(X_train.shape[1]),
    "train_size": int(X_train.shape[0]),
    "test_size_rows": int(X_test.shape[0]),
    "recommended_metrics": cfg.get("recommended_metrics", []),
}
```

Ten blok tworzy metadane, czyli opis przygotowanego zbioru. To nie jest tylko ozdoba. Dzięki temu później wiadomo, jaki był task, ile było cech po preprocessingu i jakie metryki miały sens dla danego problemu.

```
np.savez_compressed(
    npz_path,
    X_train=prepared.X_train,
    X_test=prepared.X_test,
    y_train=prepared.y_train,
    y_test=prepared.y_test,
)
```

Na końcu dane są zapisywane do gotowych plików. To rozdziela etap przygotowania danych od etapu modelowania. Osoba od sieci albo eksperymentów nie musi już wracać do CSV i nie ma pokusy, żeby robić własny preprocessing po swojemu.

3.3 src/run_data_prep.py

Ten plik jest małym uruchamiaczem do warstwy danych. Jego rola jest prosta: z terminala wywołać przygotowanie danych dla wskazanego tasku.

```
parser = argparse.ArgumentParser(description="Przygotowanie danych dla wybranego tasku.")
parser.add_argument("--task", required=True)
parser.add_argument("--test-size", type=float, default=0.20)
parser.add_argument("--random-state", type=int, default=42)
parser.add_argument("--no-scale", action="store_true")

loader = ProjectDataLoader()
prepared = loader.prepare(...)
paths = loader.save_prepared_dataset(prepared)
```

Z punktu widzenia projektu ten plik robi ważną rzecz organizacyjną: zamienia złożony pipeline na jedną komendę. Dzięki temu inni członkowie grupy nie muszą grzebać w środku data_loadera, tylko po prostu uruchamiają przygotowanie danych.

3.4 src/eda.py

To jest prosty moduł do wstępnej analizy danych. Jego zadanie nie polega na budowie modeli, tylko na szybkim zrozumieniu, z czym w ogóle pracuje projekt.

```
summary = {
    "shape": {"rows": int(df.shape[0]), "columns": int(df.shape[1])},
    "missing_values": {k: int(v) for k, v in df.isna().sum().to_dict().items()},
    "divorce_status_distribution": df["Divorce_Status"].value_counts(normalize=True).round(4).to_dict(),
    "marital_satisfaction_distribution": df["Marital_Satisfaction"].value_counts(normalize=True).round(4).to_dict(),
}
```

Najpierw tworzysz podsumowanie zbioru: rozmiar, typy kolumn, braki danych i rozkład najważniejszych targetów. To pozwala od razu zobaczyć, że klasyfikacja jest niezbalansowana.

```
fig_specs = [
    ("Divorce_Status", "bar", "divorce_status_balance.png"),
    ("Marital_Satisfaction", "bar", "marital_satisfaction_balance.png"),
    ("Children_Count", "hist", "children_count_hist.png"),
    ("Years_Since_Marriage", "hist", "years_since_marriage_hist.png"),
]
```

Drugi blok generuje proste wykresy. One nie są ozdobą, tylko wstępną kontrolą sensowności danych i trudności problemu.

4. Część 2 - UM

4.1 src/ml_config.py

W części 2 Twoja rola znowu była infrastrukturalna. Ten plik opisuje eksperymenty UM: kto za co odpowiada, jaki parametr jest strojony i jakie wartości mają być sprawdzone.

```
@dataclass(frozen=True)
class ExperimentDefinition:
    experiment_id: str
    task_name: str
    model_key: str
    owner: str
    tuned_param: str
    values: list
    fixed_params: dict
    primary_metric: str
```

Najpierw definiujesz jednolity format eksperymentu. Dzięki temu cały kod części 2 może działać na jednej strukturze danych zamiast na luźnych słownikach.

```

UM_EXPERIMENTS = [
    ExperimentDefinition(
        experiment_id="classification_divorce_decision_tree_max_depth",
        task_name="classification_divorce",
        model_key="decision_tree_classification",
        owner="person2",
        tuned_param="max_depth",
        values=[3, 5, 10, 20],
        primary_metric="balanced_accuracy",
    ),
    ExperimentDefinition(
        experiment_id="regression_children_knn_n_neighbors",
        task_name="regression_children",
        model_key="knn_regression",
        owner="person3",
        tuned_param="n_neighbors",
        values=[3, 5, 11, 21],
        primary_metric="rmse",
    ),
]

```

Lista eksperymentów jest tak naprawdę planem pracy zakodowanym w Pythonie. Zawiera metodę, task, właściciela, parametr strojony i cztery wartości parametru. To jest ważne, bo wymagania projektu mówiły o testowaniu wielu wartości parametrów osobno dla klasyfikacji i regresji.

4.2 src/ml_models.py

Ten plik buduje rejestr modeli klasycznego uczenia maszynowego. Chodzi o to, żeby runner eksperymentów nie musiał znać szczegółów każdej biblioteki osobno.

```

@dataclass(frozen=True)
class ModelSpec:
    method_name: str
    family: str
    owner: str
    problem_type: str
    factory: EstimatorFactory
    default_params: dict

```

ModelSpec opisuje model na poziomie organizacyjnym: jaka to metoda, do jakiej rodziny należy, kto ją prowadzi i jak ją stworzyć.

```

MODEL_REGISTRY = {
    "decision_tree_classification": ModelSpec(... factory=_decision_tree_classifier ...),
    "random_forest_regression": ModelSpec(... factory=_random_forest_regressor ...),
    "knn_classification": ModelSpec(... factory=_knn_classifier ...),
    "gradient_boosting_regression": ModelSpec(... factory=_gb_regressor ...),
}

```

Rejestr modeli spina nazwy logiczne używane w konfiguracji z rzeczywistymi klasami ze sklearn. Dzięki temu w eksperymentach nie ma if-ów dla każdego modelu, tylko jest jedno wspólne API.

4.3 src/ml_metrics.py

Ten plik zbiera metryki dla klasyfikacji i regresji. Jego sens jest prosty: wszystkie metody UM są oceniane tym samym zestawem wskaźników.

```
def evaluate_classification(y_true, y_pred):
    return {
        "accuracy": float(accuracy_score(y_true, y_pred)),
        "balanced_accuracy": float(balanced_accuracy_score(y_true, y_pred)),
        "precision": float(precision_score(y_true, y_pred, zero_division=0)),
        "recall": float(recall_score(y_true, y_pred, zero_division=0)),
        "f1": float(f1_score(y_true, y_pred, zero_division=0)),
    }
```

Dla klasyfikacji nie ograniczasz się do accuracy, bo zbiór jest niezbalansowany. Dlatego balanced_accuracy, precision, recall i f1 są tutaj ważniejsze niż sama skuteczność ogólna.

```
def evaluate_regression(y_true, y_pred):
    mse = float(mean_squared_error(y_true, y_pred))
    return {
        "mse": mse,
        "rmse": float(np.sqrt(mse)),
        "mae": float(mean_absolute_error(y_true, y_pred)),
        "r2": float(r2_score(y_true, y_pred)),
    }
```

Dla regresji używasz zestawu standardowych metryk: błąd średniokwadratowy, jego pierwiastek, błąd bezwzględny i R2.

4.4 src/ml_experiments.py

To jest centralny plik części 2. Łączy dane, konfigurację eksperymentów, rejestr modeli, metryki i zapis wyników.

```
def load_processed_dataset(task_name):
    npz_path = PROCESSED_DATA_DIR / f"{task_name}.npz"
    metadata_path = PROCESSED_DATA_DIR / f"{task_name}_metadata.json"
    ...
    data = np.load(npz_path)
    metadata = json.load(open(metadata_path, "r", encoding="utf-8"))
    return data["X_train"], data["X_test"], data["y_train"], data["y_test"], metadata
```

Najpierw runner wczytuje wcześniej przygotowane dane. To jest bardzo ważne, bo część 2 nie robi własnego preprocessingu od nowa, tylko korzysta dokładnie z tych samych splitów, co część 1. Dzięki temu porównanie SSN i UM jest uczciwe.

```
for value in definition.values:
    for repeat_idx, seed in enumerate(self.seeds, start=1):
```

```

params = dict(model_spec.default_params)
params.update(definition.fixed_params)
params[definition.tuned_param] = value
estimator = model_spec.factory(**params)
estimator.fit(X_train, y_train.reshape(-1))
y_train_pred = estimator.predict(X_train)
y_test_pred = estimator.predict(X_test)

```

To jest serce pętli eksperymentalnej. Dla każdej wartości strojącego parametru i dla każdego powtórzenia tworzysz model, uczysz go na train, a potem liczysz predykcje na train i test. Dzięki temu można potem porównywać średnie wyniki.

```

row = {
    "experiment_id": definition.experiment_id,
    "task_name": definition.task_name,
    "problem_type": problem_type,
    "owner": definition.owner,
    "method_name": model_spec.method_name,
    "tuned_param": definition.tuned_param,
    "param_value": value,
    "repeat": repeat_idx,
    "random_seed": seed,
}

```

Każdy przebieg jest zapisywany jako jeden rekord tabeli wynikowej. To pozwala później policzyć średnie, robić wykresy i odtworzyć cały eksperyment.

```

grouped = df.groupby("param_value", dropna=False).mean(numeric_only=True).reset_index()
summary_path = self.results_dir / f'{definition.experiment_id}_summary.csv'
grouped.to_csv(summary_path, index=False)

```

Po surowych wynikach tworzysz też summary, czyli uśrednione wyniki dla każdej wartości parametru. To jest materiał, który łatwo wstawić do raportu.

```

plt.plot(grouped["param_value"], grouped[train_col], marker="o", label=f"train_{metric}")
plt.plot(grouped["param_value"], grouped[test_col], marker="o", label=f"test_{metric}")
plot_path = self.plots_dir / f'{definition.experiment_id}_{metric}.png'

```

Runner generuje też wykresy. Dzięki temu osoba pisząca raport nie musi od zera obrabiać wyników numerycznych.

```

def select_definitions(owner=None, task_name=None):
    selected = UM_EXPERIMENTS
    if owner:
        selected = [d for d in selected if d.owner == owner]
    if task_name:
        selected = [d for d in selected if d.task_name == task_name]
    return selected

```


Ten blok pozwala uruchamiać tylko część eksperymentów, np. tylko dla jednej osoby albo tylko dla jednego tasku. To było praktyczne w pracy zespołowej, bo każdy mógł odpalać swój zakres bez mieszania się w cudze wyniki.

4.5 run_um_experiments.py

```
from src.ml_experiments import main

if __name__ == "__main__":
    main()
```

To jest mały plik startowy. Z technicznego punktu widzenia robi niewiele, ale organizacyjnie jest ważny: daje jedno proste wejście do części 2 bez konieczności odpalania modułu głębiej z katalogu src.

5. Jak Twoja część łączy się z resztą projektu

- Osoba 2 pisała główną implementację sieci neuronowej od zera. Jej kod korzystał z danych przygotowanych przez Twój pipeline.
- Osoba 3 robiła eksperymenty SSN i część metod UM. Korzystała z ustalonych przez Ciebie danych wejściowych, nazw tasków i formatów wyników.
- Osoba 4 spinała raport i porównania. Mogła to zrobić dlatego, że Twoja warstwa zapewniała spójne dane i powtarzalne wyniki.

Najkrócej: Twoja część nie trenowała modelu dla sportu. Ona ustalała warunki eksperymentu. To dzięki temu obie części projektu mogły być porównywane na tych samych danych.

6. Najkrótsze streszczenie dla osoby od podsumowania

Kod Osoby 1 odpowiada za wspólną infrastrukturę projektu. W części SSN przygotowuje dane: waliduje kolumny, wybiera target, robi preprocessing, dzieli dane na train i test, zapisuje gotowe macierze i metadane oraz wykonuje prostą analizę EDA. W części UM buduje plan eksperymentów, rejestr metod, wspólne metryki oraz runner, który uruchamia modele na tych samych przygotowanych danych i zapisuje wyniki oraz wykresy. Dzięki tej warstwie pozostali autorzy mogli dopisywać swoje modele i eksperymenty bez zmiany architektury projektu.

7. Co trzeba z tego naprawdę zrozumieć

- config.py mówi, jakie taski w ogóle istnieją i jak je traktować;
- data_loader.py zamienia surowy CSV w gotowe macierze do modelowania;
- run_data_prep.py pozwala przygotować dane jedną komendą;
- eda.py daje szybki obraz trudności danych;
- ml_config.py definiuje eksperymenty dla części 2;
- ml_models.py wiąże nazwy metod z modelami ze sklearn;

- `ml_metrics.py` ocenia predykcje;
- `ml_experiments.py` wykonuje eksperymenty, zapisuje wyniki i wykresy;
- `run_um_experiments.py` jest prostym wejściem z terminala.

Jeżeli nowa osoba zrozumie te dziewięć punktów, to spokojnie napisze sensowne podsumowanie Twojej części projektu.